

More about if-else

CMSC 104 Section 2
16 September 2026

Administrative Notes

Quiz 1 is in class next Monday

- Bring a writing implement - pen or pencil!

Classwork 3 is due next Monday, 21 September

Project 1 is due next Wednesday, 23 September

A little more about the if-else statement

We snuck that into Classwork 3; you need to know about it.

Remember that the key is having a boolean expression

- Something that will evaluate to either True or False
 - This is actually a lie we're going to talk more about in a few weeks, but just roll with me here
- It can be a single Boolean variable
 - `is_prime = True`
- It can be an expression that evaluates to either True or False
 - `num == 6`

Notation

If I didn't make it clear in a previous lecture:

= means an assignment statement

Take the value on the right of the = sign; and put it in the location in memory associated with the variable on the left.

s = 6 assigns the value 6 to the variable s; Python puts the value 6 in the location in memory associated with s in the symbol table

== is a comparison for equality

If you want to find out if Python regards two different things as being equal, this is how you do it. You always get back True or False

school == "UCLA" returns True or False

y + 6 == x*3

Let's start with if-else

You want to do one of two different things depending on a condition:

- Do one thing (or set of things) if a condition is True
- Do something else (or some other set of things) if that same condition is False

Part 4 of Classwork 3

- `is_prime == True` # we start by assuming that a number is prime until proven otherwise
- Now, check to see if that number is evenly divisible by other numbers.
 - A number, `x`, is evenly divisible by another number, `y`, if `x%y == 0`.
 - That is, if there is no remainder; the modulus is zero, then `y` divides `x`.
 - So that would mean that `x` isn't prime.
- Now we can build a conditional - an if-else:

```
if x%y == 0:  
    is_prime = False  
else:  
    print("no division")
```

Syntax & semantics:

```
if boolean_condition:
    """
    one or more lines
    of code
    """
else:
    """
    one or more lines
    of code
    """
```

If the `boolean_condition` evaluates to `True`, execute all of the indented lines of code between `if` and `else`. Skip all the code after `else`. Go to the next un-indented line of code.

If the `boolean_condition` evaluates to `False`, skip all of the indented lines of code between `if` and `else`. Execute all the indented lines of code after `else`. Then continue with the next un-indented line if there is one.

Wait - that seems kind of fake

You didn't really need that code after "else" did you?

Correct - "else" is optional

Sometimes you want to do one thing (or set of things) if a condition is True. If the condition is False you don't want to do anything.

if without else: Syntax & semantics:

```
if boolean_condition:  
    """  
    one or more lines  
    of code  
    """
```

If the `boolean_condition` evaluates to `True`, execute all of the indented lines of code after `if`. Continue with the next un-indented line of code if there is one.

If the `boolean_condition` evaluates to `False`, do nothing. Continue with the next un-indented line if there is one.

For Classwork 3:

```
if x%y == 0:  
    is_prime = False
```

We presumed x is prime. If y does not evenly divide x, that presumption might still be good. So there's nothing to do

Putting it in a loop

The code on the previous slide let us check to see if x was divisible by one specific number.

To check if x is truly prime, we need to check to see if x is divisible by any of the numbers between 2 and the square root of x . Assuming that we have imported the `math` library:

```
is_prime = True

for num in range(2, math.sqrt(x)):
    if x%num == 0:
        is_prime = False
```

Now you can proceed with the rest of the problem.